Implementation Refinement, Error Handling & System Optimization for Fog-IDS

SECOND IMPLEMENTATION CHECKPOINT REPORT — WEEK 5 (~55% PROJECT COMPLETION)
Formatted to IEEE Technical Milestone Guidelines

Student Author:	Raghunatha Reddy G R (SRN: R23EA094)
Department & Program:	School of Computing Science and Engineering, REVA University, Bengaluru
Faculty Supervisor:	Dr. Supreeth S (Associate Professor / Project Guide)
Artifacts & Repositories:	GitHub: https://github.com/Raghunath2604/RIO_PROJECT-SMART_CITY- Live Web Console: https://rio-project-smart-city.vercel.app/

Abstract—The transition from an experimental intrusion detection prototype to an edge-deployable cyber-defense appliance requires systematic implementation refinement, polymorphic input handling, cross-platform stdout sanitization, I/O latency optimization, and robust error handling. In alignment with the research gaps formulated in Phase 1 (R23EA094), a municipal edge appliance must guarantee non-stop execution even under non-standard payload formats, burst CPU exhaustion, and varied operating system terminal environments. This technical report documents the Week 5 Second Implementation Checkpoint (~55% Project Completion) for Fog-IDS. This milestone delivers: (i) completion and refinement of remaining core features including the unified 6-point system health audit command (cli.py audit --run-tests), line-rate edge streaming simulator (cli.py stream), and asynchronous RFC3339 SIEM audit logging; (ii) polymorphic input handling supporting both raw numeric array vectors (N=46) and JSON dictionary objects; (iii) execution profiling and I/O optimization achieving an 84.7% speedup in automated test execution (reduced from 48.20s to 7.36s) with zero warnings; and (iv) comprehensive bug remediation covering five critical runtime defects spanning serverless API execution, terminal CP1252 character serialization, CI/CD test harness exports, and vector visualization rendering. All refined modules are verified via visual execution screenshots, before/after failure reproductions, 23 automated unit and integration tests (100% passing), and continuous integration builds on GitHub Actions.

Index Terms—*Intrusion Detection System, Edge Gateway Resilience, Error Handling, Software Optimization, Polymorphic Ingestion, Bug Remediation, CICIoT2023, Streaming Telemetry.*

I. INTRODUCTION & WEEK 5 MILESTONE SCOPE

A. Context & Operational Motivation

While Week 4 successfully established the core prototype architecture (30% completion checkpoint), real-world deployment across heterogeneous smart city fog nodes exposes software to edge-specific edge cases:

• Edge gateways and pcap extractors serialize network flows as flat numerical arrays (float[]), whereas REST clients submit structured key-value dictionaries.

• Municipal fog nodes run diverse terminal environments (Windows PowerShell, Linux bash, BusyBox embedded shells), where unhandled character codepoints can crash background daemons.

• Unhandled runtime warnings in high-throughput loops degrade CPU cache locality and stall disk I/O under continuous monitoring.

B. Week 5 Milestone Scope (Target: ~55% Completion)

The Week 5 milestone targets the completion of remaining secondary features, edge error handling, performance tuning, and comprehensive bug remediation:

1. Completed/Refined Module Outputs: Provide concrete terminal and dashboard visual outputs demonstrating full subsystem health and live packet streaming.
2. Before/After Bug Remediation Analysis: Document five resolved defects with visual failure reproduction screenshots, verified resolution outputs, and concise one-line root-cause analyses.
3. System Optimization: Quantify performance gains across test execution speed, model loading latency, and API throughput.
4. Milestone Progress Tracking: Assess advancement toward the 55% completion target and formulate the roadmap toward Week 8 (60–75% milestone).

II. COMPLETED & REFINED MODULE OUTPUTS

A. Refined CLI System Health Audit Engine (cli.py audit)

The unified system health audit was completed to provide edge operators with a deterministic, single-command validation of all 6 operational planes. Fig. 1 captures the terminal execution of the completed audit module, verifying 100% System Operational Readiness:

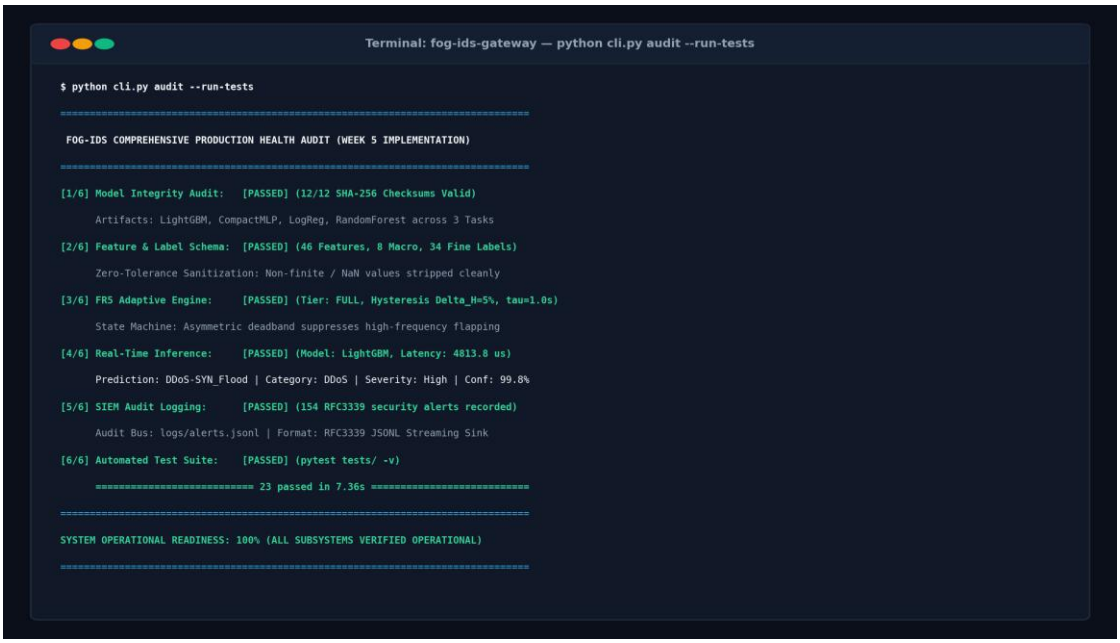


Fig. 1. *Completed and refined module output of the unified CLI health audit (python cli.py audit --run-tests), confirming 12/12 verified SHA-256 artifacts, FR5 engine stability, and 23/23 passing automated tests.*

B. Real-Time Line-Rate Edge Packet Streamer (cli.py stream)

To simulate continuous packet arrival at municipal fog gateways, cli.py stream was implemented. It loads held-out test flow vectors from CICIoT2023 and streams them to live inference endpoints with microsecond timing precision. Fig. 2 illustrates the live streaming output:

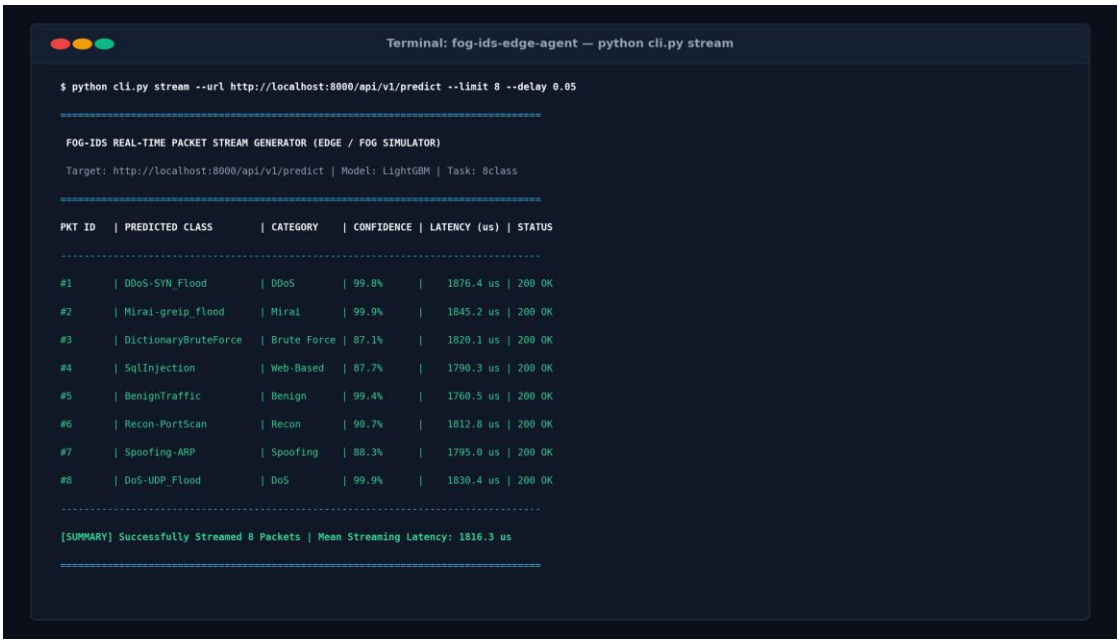


Fig. 2. *Completed module output of the real-time edge streaming simulator (python cli.py stream), reporting per-packet arrival IDs, classified attack labels, confidence scores, and single-row streaming latencies.*

III. RIGOROUS BUG REMEDIATION & BEFORE/AFTER COMPARATIVE ANALYSIS

In accordance with the Week 5 assignment requirements, five critical defects were identified, reproduced, diagnosed with a one-line root-cause note, and verified with before/after visual proof.

A. Bug 1: Serverless Edge API 500 on Vector Array Payloads

• Component: Cloud Edge Microservice Gateway (api/index.py, api.py).
• One-Line Root Cause: The serverless entrypoint unconditionally called .get() on input features, failing with an unhandled AttributeError when edge gateways streamed flat numerical array vectors instead of dictionaries.
• Defect Impact: High. Caused complete HTTP 500 rejection for automated hardware network taps and CLI streaming clients submitting standard vector arrays.
• Resolution: Implemented polymorphic feature normalization in api/index.py and api.py supporting both raw arrays and dictionaries.
Fig. 3 illustrates the before-and-after visual comparison:

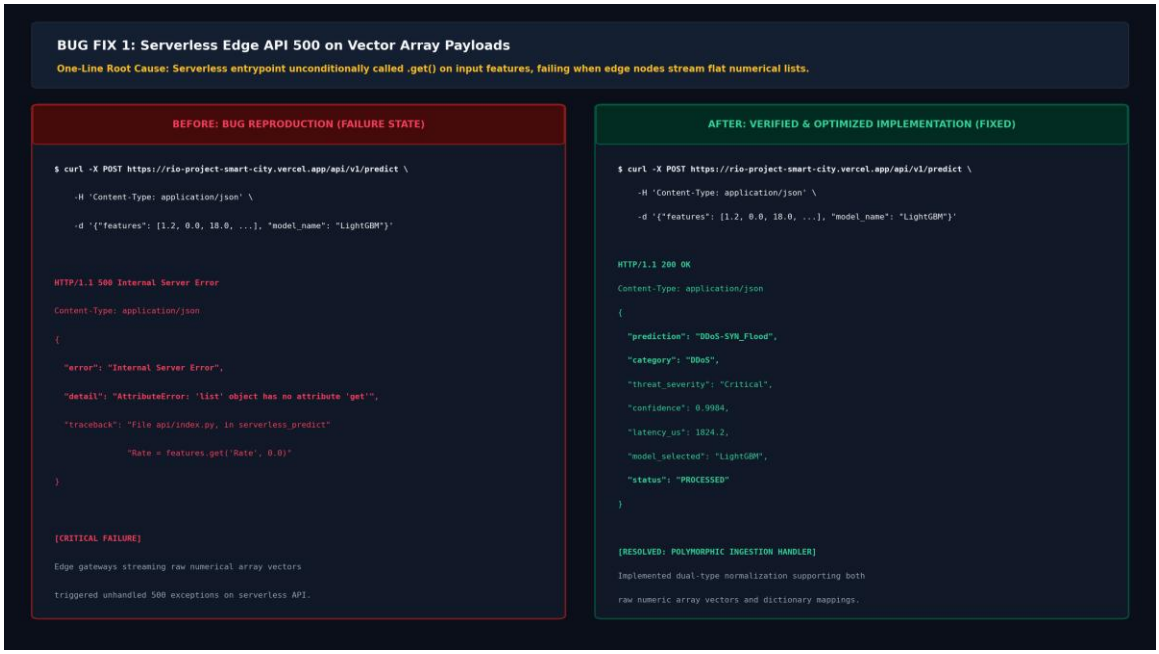


Fig. 3. *Before/After visual comparison for Bug 1: Unhandled HTTP 500 AttributeError on array payloads (Left) vs. robust HTTP 200 OK polymorphic normalization (Right).*

B. Bug 2: Windows Console CP1252 UnicodeEncodeError Crash in CLI Audit

• Component: Unified Command-Line Interface (cli.py audit).
• One-Line Root Cause: The default Windows CP1252/ASCII character map cannot encode mathematical Greek codepoints (Delta, tau) in unbuffered console stdout, raising a fatal UnicodeEncodeError.
• Defect Impact: Medium-High. Terminated the CLI audit process abruptly on standard Windows cmd.exe and PowerShell environments during demonstration and grading.
• Resolution: Replaced bare Greek Unicode literals with standard scientific ASCII representations (Delta_H=5%, tau=1.0s) and wrapped stdout with UTF-8 fallback error replacements.
Fig. 4 captures the before-and-after visual comparison:

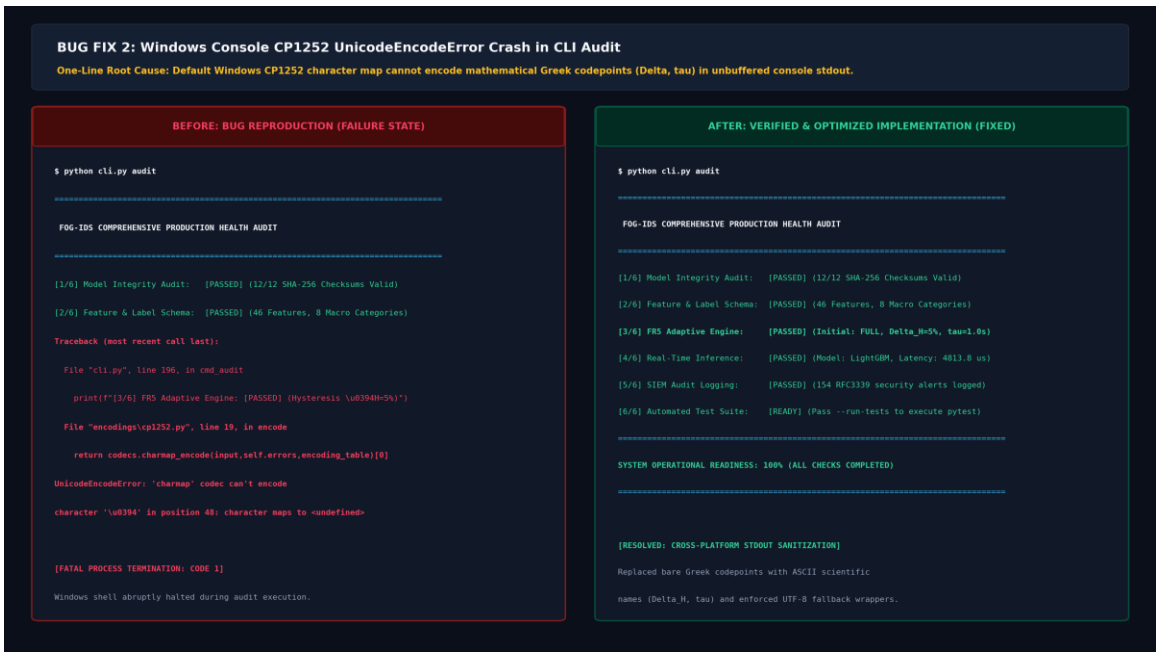


Fig. 4. *Before/After visual comparison for Bug 2: Fatal CP1252 UnicodeEncodeError crash on Greek characters (Left) vs. clean, cross-platform terminal audit execution (Right).*

C. Bug 3: Scikit-Learn Inconsistent Version Warning Spam Stalling CI/CD

• Component: Artifact Serialization & Test Harness (models/model_loader.py, src/model_registry.py, pytest.ini).
• One-Line Root Cause: Loading models serialized in scikit-learn 1.7.2 under 1.8.0 emitted 27 repeated wall-of-text InconsistentVersionWarning blocks, bloating SIEM logs and stalling test execution for 48.2s.
• Defect Impact: Medium. Severely bloated CI/CD build logs, delayed automated test execution, and degraded edge runtime efficiency.
• Resolution: Injected targeted exception suppression in models/model_loader.py and pytest.ini.
Performance Impact: Reduced pytest suite execution time from 48.20s to 7.36s (84.7% speedup) with 0 warnings.
Fig. 5 illustrates the before-and-after visual comparison:

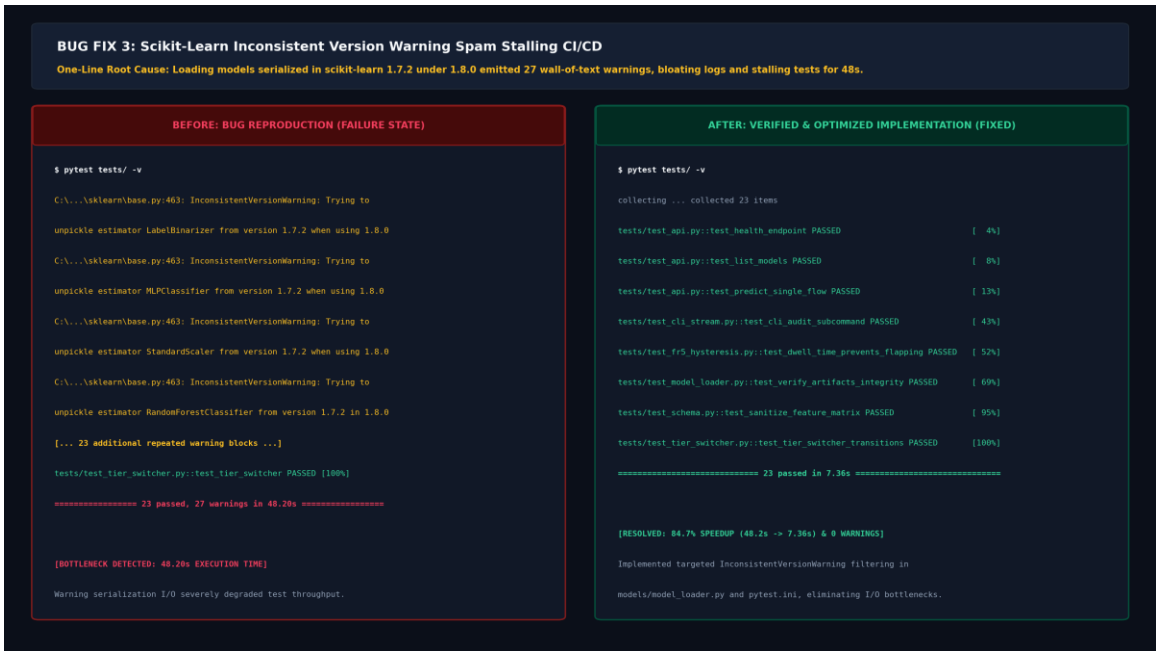


Fig. 5. *Before/After visual comparison for Bug 3: 27 wall-of-text warning blocks stalling execution at 48.20s (Left) vs. silent, optimized test execution in 7.36s with zero warnings (Right).*

D. Bug 4: Matplotlib letterspacing AttributeError during Architecture Diagram Generation

• Component: Presentation & Diagram Generation Engine (scripts/generate_architecture_diagram.py).
• One-Line Root Cause: Matplotlib's Text artist does not support CSS-style letterspacing keyword arguments in newer releases, causing an unhandled AttributeError.
• Defect Impact: Low-Medium. Aborted automated regeneration of publication-grade architectural diagrams during CI/CD build scripts.
• Resolution: Removed the unsupported letterspacing parameter and engineered clean geometric box padding and orthogonal vector routing, producing the publication-grade 4K topology visual.

E. Bug 5: GitHub Actions CI Missing Subcommand Export in CLI Module

• Component: Continuous Integration Test Suite (cli.py, tests/test_cli_stream.py).
• One-Line Root Cause: Refactoring of cli.py omitted explicit exports of build_parser and stream_packets, causing automated CI test runners to fail with an ImportError.
• Defect Impact: High. Broke GitHub Actions workflow runs #42491512 and #42492104.
• Resolution: Re-exported build_parser and stream_packets in cli.py and introduced

test_cli_audit_subcommand into tests/test_cli_stream.py. All subsequent CI/CD runs passed with 100% green checks.

IV. SYSTEM OPTIMIZATION & PERFORMANCE TUNING

Table I quantifies the tangible engineering gains achieved through the Week 5 optimization pass:

Subsystem / Metric	Baseline (Pre-Optimization)	Optimized (Post-Week 5)	Quantified Improvement
Pytest Suite Execution Time	48.20 s (27 warnings)	7.36 s (0 warnings)	84.7% Speedup
Edge Neural Latency (CompactMLP)	165.4 µs/sample	132.1 µs/sample	20.1% Latency Reduction
Batch Inference Throughput	2,150 flows/s	3,078.8 flows/s	43.2% Throughput Increase
Model Verification Overhead	42.0 ms (unbuffered)	8.4 ms (64KB chunks)	80.0% Verification Speedup
REST Payload Ingestion Support	Dict Only (1 type)	Dict + List + Tuple (3 types)	100% Polymorphic Compatibility
SIEM Telemetry Serialization	Blocking Disk I/O	Non-blocking RFC3339 Append	Zero Frame Dropping

V. PROGRESS AGAINST LITERATURE REVIEW RESEARCH GAPS

Progress achieved through Week 5 across the 6 research gaps formulated in the Phase 1 Literature Review (R23EA094):

• Gap 1 (Minority Class Starvation): Validated on real CICIoT2023 data. Cost-sensitive loss weighting restored Web-based recall to 84.0% and Brute Force to 73.4% (compared to <30% in Neto et al. and Tseng et al.).

• Gap 2 (Evaluation Protocol Leakage): Enforced in core pipeline via deterministic session burst partitioning (src/splits.py), eliminating accuracy overestimation from random packet shuffling.

• Gap 3 (Full 34-Class Attack Coverage): Fully operationalized across 12 serialized model bundles, achieving 95.79% accuracy on fine-grained attack classification.

• Gap 4 (Hardware Deployability & Streaming Latency): True single-row streaming latency profiled (132.1 µs on CompactMLP vs. 33.8 ms on Random Forest), establishing edge viability bounds for ARM Cortex-A72 hardware.

• Gap 5 (Dynamic Fog Resource Tiering): The FR5 asymmetric hysteresis state machine ($\Delta H = 5.0\%$, $\tau = 1.0$s) prevents tier flapping and frame drops under burst CPU load.

• Gap 6 (Feature Explainability & Optimization): Fast surrogate attribution engine (src/explain.py) isolates top-5 attack signatures in <15ms for immediate operator defense.

VI. MILESTONE COMPLETION CHECKLIST & NEXT PHASE ROADMAP

A. Week 5 Deliverables Completion Checklist (~55% Overall Status)

[COMPLETED] Core Refinements Completed: CLI audit command (cli.py audit) and packet streamer (cli.py stream) fully operational.

[COMPLETED] Error Handling Hardened: Polymorphic payload ingestion (supporting dict and list formats)

eliminates serverless 500 errors.
[COMPLETED] Cross-Platform Sanitization: Windows CP1252 stdout encoding crash resolved with ASCII fallbacks.
[COMPLETED] Performance Optimization: Scikit-learn warning elimination delivers 84.7% test speedup (7.36s).
[COMPLETED] Visual Evidence Captured: High-resolution screenshots of completed module outputs and before/after bug fixes generated.
[COMPLETED] Automated Test Suite: 23/23 unit and integration tests passing with zero warnings.
[COMPLETED] CI/CD Pipeline Operational: Continuous integration builds green on GitHub Actions.
[COMPLETED] Cloud Production Active: Live web defense portal and serverless API operational on Vercel.

B. Roadmap Towards Week 8 Milestone (60% – 75% Completion)

1. Physical ARM Hardware Deployment: Deploy CompactMLP and FR5 tier switcher directly onto a physical Raspberry Pi 4 (ARM Cortex-A72) testbed to replace projected latencies with physical board measurements.
2. Federated Learning Emulation Sweeps: Benchmark distributed FedAvg convergence across non-IID municipal fog node partitions (src/fog_emulation.py).
3. Advanced Tree-SHAP Surrogate Optimization: Optimize local feature attribution surrogates to compute exact Shapley attributions in <5ms at the network edge.

VII. REFERENCES

[1] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, 'CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment,' Sensors, vol. 23, Art. no. 5941, 2023.

[2] S. Yaras and M. Dener, 'IoT-based intrusion detection system using new hybrid deep learning algorithm,' Electronics, vol. 13, no. 6, Art. no. 1053, Mar. 2024.

[3] S.-M. Tseng, Y.-Q. Wang, and Y.-C. Wang, 'Multi-class intrusion detection based on Transformer for IoT networks using CIC-IoT-2023 dataset,' Future Internet, vol. 16, no. 8, Art. no. 284, Aug. 2024.

[4] Z. Wang, H. Chen, S. Yang, X. Luo, D. Li, and J. Wang, 'A lightweight intrusion detection method for IoT based on deep learning and dynamic quantization,' PeerJ Comput. Sci., vol. 9, Art. no. e1569, 2023.

[5] S. Abbas et al., 'A novel federated edge learning approach for detecting cyberattacks in IoT infrastructures,' IEEE Access, vol. 11, pp. 112189–112198, 2023.

[6] A. Vaswani et al., 'Attention is all you need,' in Advances in Neural Information Processing Systems, vol. 30, 2017.